AWS VPC PEERING

Private Cross-VPC Communication with a Bastion Host

EWB CLOUD COMPUTING INTERNSHIP PROJECT

Student	Tanay Jain
AICTE Student ID	STU970f6f93295d91789482176
College	Birla Institute of Technology, Mesra (Jaipur Campus)
Internship	Amazon Web Services (AWS) Cloud Computing Internship (EWB Edutech Pvt. Ltd.)
AWS Region	ap-south-1 (Mumbai)
Completion date	19 October 2026

Evidence-based implementation report

This report distinguishes observed test results from verification gaps.

1. Executive Summary

This project builds private IPv4 connectivity between two Amazon Virtual Private Clouds (VPCs) in the Mumbai Region using an AWS VPC peering connection. VPC A contains a public bastion host and one private application host. VPC B contains a second private application host and has no Internet Gateway. Route tables and security-group rules enable narrowly scoped SSH administration and ICMP connectivity between the private workloads.

The supplied terminal evidence records successful SSH access to the bastion, App-A and App-B using OpenSSH ProxyJump; forward and reverse ping tests each received 4 of 4 replies with 0% packet loss; and a controlled ICMP rule-removal test caused ping to fail before it recovered after the rule was restored. Cleanup evidence shows all three lab EC2 instances in Terminated state and the VPC console showing only the default VPC in the filtered list.

> **Project status and evidence integrity**
>
> The original pre-peering baseline ping failure was not captured before peering was activated and has not been fabricated retrospectively. The controlled post-peering negative test and recovery are included as separate, genuine evidence. The supplied cleanup screenshots do not include an EBS-volumes, Elastic IPs, or billing page, so this report does not claim that the account has zero residual charges.

2. Goals and Scope

- Create two VPCs with non-overlapping CIDRs and explicit subnet-to-route-table associations.
- Keep both application instances private, with no public IPv4 addresses.
- Provide administration through a single public bastion using SSH ProxyJump.
- Establish VPC peering and configure forward and return routes for both private CIDR ranges.
- Prove private reachability in both directions and demonstrate how removing the destination ICMP rule affects traffic.
- Preserve the actual console and terminal screenshots, then document cleanup.

3. Architecture Overview

The architecture below represents the working configuration during validation, before cleanup. The peering connection was Active at that time. The later cleanup evidence confirms that the project EC2 instances were terminated and the project VPCs were removed.

Host: AWS architecture | Test: Design overview | Result: Shows VPC A, VPC B, subnet ranges, bastion path, peering ID, Internet Gateway ID and all three route tables.

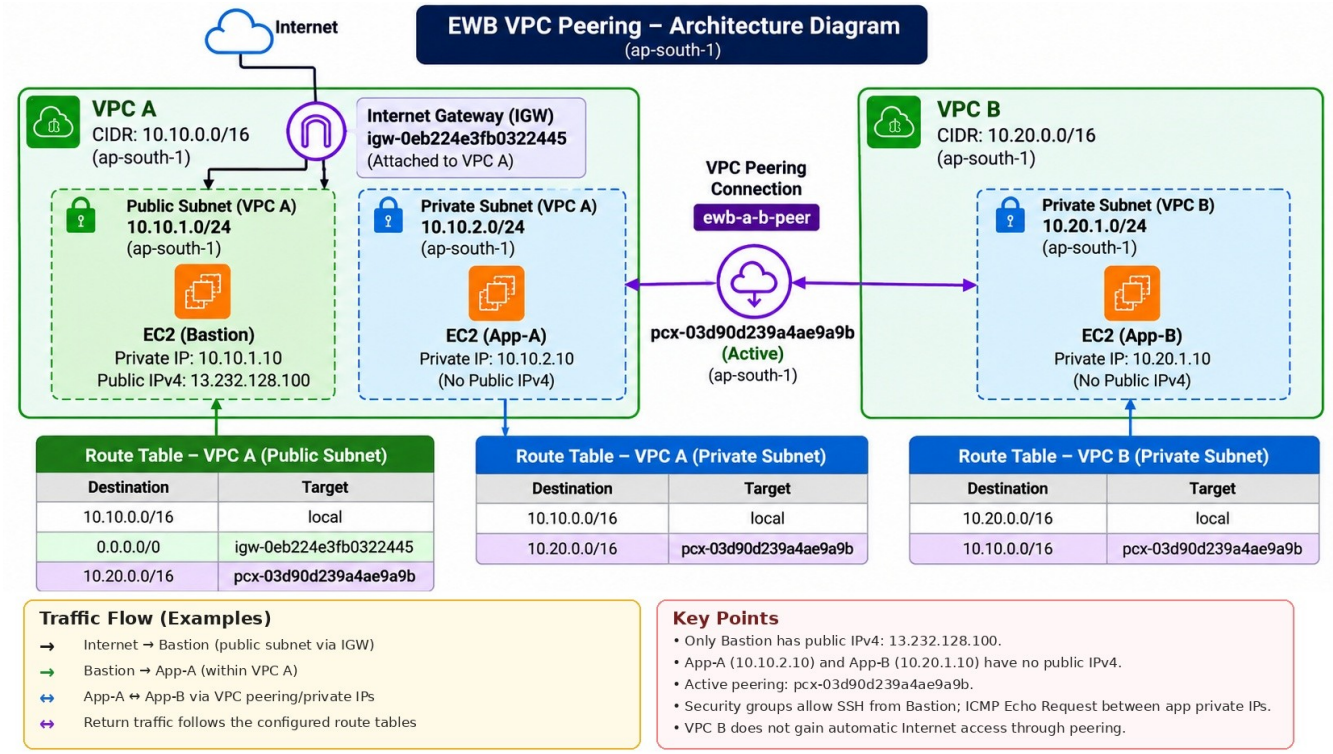


Figure 1. As-built architecture at test time; resource state changed during the documented cleanup.

4. Network Address Plan and Resource Inventory

All identifiers below were transcribed from the supplied AWS Console screenshots or explicitly provided by the student. These IDs describe resources during the test run; the cleanup screenshots later show the instances terminated and project VPCs removed.

4.1 VPCs

Logical network	AWS name	VPC ID	IPv4 CIDR	Lifecycle evidence
VPC A	ewb-vpc-a	vpc-0a6ce73ff02b1503f	10.10.0.0/16	Deleted during cleanup; active during tests
VPC B	ewb-vpc-b	vpc-075df51206347c58f	10.20.0.0/16	Deleted during cleanup; active during tests

4.2 Subnets and explicit associations

Subnet role	Name	Subnet ID	CIDR	VPC / AZ	Associated route table
Public subnet A	ewb-a-public	subnet-0484e07086d26edb5	10.10.1.0/24	VPC A / ap-south-1a	rtb-07eb6affd6f3a0e73
Private subnet A	ewb-a-private	subnet-0d7771459ff9a714d	10.10.2.0/24	VPC A / ap-south-1a	rtb-0e0dde41748dfa436
Private subnet B	ewb-b-private	subnet-0b71be209ac84f3f3	10.20.1.0/24	VPC B / ap-south-1a	rtb-0dfff1ec80beab2f9

4.3 Gateways and peering

Resource	Name	Resource ID	Observed state / purpose
Internet Gateway	ewb-igw-a	igw-0eb224e3fb0322445	Attached to VPC A; public-subnet default route
VPC peering	ewb-a-b-peer	pcx-03d90d239a4ae9a9b	Active during validation; deleted with the project network during cleanup

4.4 EC2 instances

Instance	Instance ID	Subnet	Private IPv4	Public IPv4	Type / AZ	Cleanup state
ewb-bastion	i-0a96fe6409821ab86	VPC A / ewb-a-public	10.10.1.10	13.232.128.100	t3.micro / ap-south-1a	Terminated; shown Running during successful tests
ewb-app-a	i-07875dafbce902beb	VPC A / ewb-a-private	10.10.2.10	None	t3.micro / ap-south-1a	Terminated; shown Running during successful tests
ewb-app-b	i-077af8fdd52fd6b4a	VPC B / ewb-b-private	10.20.1.10	None	t3.micro / ap-south-1a	Terminated; shown Running during successful tests

4.5 Route tables and destinations

Route table	Name	Route-table ID	Explicit association	Routes shown active during validation
Public route table A	ewb-rt-a-public	rtb-07eb6affd6f3a0e73	ewb-a-public	10.10.0.0/16 → local; 0.0.0.0/0 → igw-0eb224e3fb0322445; 10.20.0.0/16 → pcx-03d90d239a4ae9a9b
Private route table A	ewb-rt-a-private	rtb-0e0dde41748dfa436	ewb-a-private	10.10.0.0/16 → local; 10.20.0.0/16 → pcx-03d90d239a4ae9a9b
Private route table B	ewb-rt-b-private	rtb-0dfff1ec80beab2f9	ewb-b-private	10.20.0.0/16 → local; 10.10.0.0/16 → pcx-03d90d239a4ae9a9b

4.6 Security groups and inbound rules

Role / name	Security group ID	Inbound protocol / port	Source CIDR
Bastion / ewb-sg-bastion	sg-0dd786e889c8dc6c9	SSH / TCP 22	49.43.176.59/32 (source shown in evidence at capture time)
App-A / ewb-sg-app-a	sg-06ebed5e7197ab5d5	SSH / TCP 22	10.10.1.10/32
App-A / ewb-sg-app-a	sg-06ebed5e7197ab5d5	Custom ICMP-IPv4 / Echo Request	10.20.1.10/32
App-B / ewb-sg-app-b	sg-0ef25547ef3d026e5	SSH / TCP 22	10.10.1.10/32
App-B / ewb-sg-app-b	sg-0ef25547ef3d026e5	Custom ICMP-IPv4 / Echo Request	10.10.2.10/32

The evidence shows default outbound allow-all rules were retained for this lab. The inbound rules use host-specific /32 sources rather than opening SSH or ICMP to 0.0.0.0/0. The bastion rule shows the public IPv4 /32 that was allowed at capture time; it is historical and should not be reused blindly after the environment has been removed.

5. Routing, Access Control, and Test Results

5.1 Route-table logic

Route table	Destination	Target	Reason
ewb-rt-a-public	10.10.0.0/16	local	VPC A internal routing
ewb-rt-a-public	0.0.0.0/0	igw-0eb224e3fb0322445	Internet path for the public bastion subnet
ewb-rt-a-public	10.20.0.0/16	pcx-03d90d239a4ae9a9b	Route from public subnet to VPC B; supports SSH path via bastion
ewb-rt-a-private	10.10.0.0/16	local	App-A internal VPC A routing
ewb-rt-a-private	10.20.0.0/16	pcx-03d90d239a4ae9a9b	Forward route for App-A → App-B
ewb-rt-b-private	10.20.0.0/16	local	App-B internal VPC B routing
ewb-rt-b-private	10.10.0.0/16	pcx-03d90d239a4ae9a9b	Return route for App-B → VPC A

5.2 Administrative SSH path

Laptop OpenSSH connects to the bastion public IPv4 13.232.128.100. The local SSH config uses ProxyJump so SSH to App-A (10.10.2.10) and App-B (10.20.1.10) is relayed through the bastion. The private hosts have no public IPv4 addresses. The `configs/ssh_config` file contains only hostnames, private IPs, username and a local key-file path; it contains no private key.

5.3 Observed test results

Test	Location / command	Observed result
Bastion SSH	Windows PowerShell: `ssh ewb-bastion`	Successful; hostname maps to private IP 10.10.1.10.
App-A SSH	Windows PowerShell: `ssh ewb-app-a`	Successful via ProxyJump; private IP 10.10.2.10.
App-B SSH	Windows PowerShell: `ssh ewb-app-b`	Successful via ProxyJump; private IP 10.20.1.10.
Forward ICMP	App-A: `ping -c 4 10.20.1.10`	4 packets received, 0% packet loss.
Reverse ICMP	App-B: `ping -c 4 10.10.2.10`	4 packets received, 0% packet loss.
Controlled negative test	App-A pinging App-B after removing App-B inbound Echo Request from 10.10.2.10/32	0 packets received; 100% packet loss shown.
Recovery	Same ping after restoring App-B inbound Echo Request from 10.10.2.10/32	4 packets received, 0% packet loss.

5.4 Baseline disclosure

Pre-peering test evidence limitation

The workbook calls for a failed ping from App-A to App-B before peering. That original pre-peering failure was not captured before the peering connection was activated. It was not recreated or presented as historical evidence. The genuine post-peering controlled negative test (19-controlled-ping-failure.PNG) and recovery test (20-controlled-ping-recovery.PNG) demonstrate that the destination security-group ICMP rule controls reachability, but they are not represented as the original baseline.

6. Mandatory Viva Questions and Answers

Why is a return route required?

VPC peering creates a network connection but does not add routes automatically. The source subnet needs a route to the peer CIDR, and the destination-side route table needs a route back to the source CIDR. Without the return route, replies may not reach the initiating host.

Why does App-B see the bastion private IP as the SSH source?

With ProxyJump, the laptop establishes one SSH connection to the bastion and the bastion establishes a separate TCP/SSH connection to App-B. App-B therefore sees the bastion ENI private address, 10.10.1.10, as the source; the security-group rule restricts TCP/22 to 10.10.1.10/32.

Why does Active peering alone not guarantee a successful ping?

Active confirms the peering connection exists. The relevant route tables must point peer CIDRs to the peering target, and security groups, network ACLs and host firewalls must allow the traffic. The destination must also be running and using the expected IP.

Why can VPC B not use VPC A's Internet Gateway over peering?

VPC peering is non-transitive and does not act as a transit gateway or internet-egress path. VPC B does not inherit Internet access from VPC A's Internet Gateway. An internet-connected design would require its own appropriate routing and egress architecture.

7. Learning Outcomes

- Planned non-overlapping VPC CIDRs and associated each subnet with the intended route table.
- Created and validated same-account, same-Region VPC peering, then added forward and return routes.
- Used inbound security groups with specific source CIDRs and kept the application instances private.
- Configured local OpenSSH ProxyJump access without copying a private key onto either private EC2 instance.
- Verified bidirectional ICMP reachability and demonstrated failure/recovery by changing only a destination ICMP rule.
- Recorded the architecture, actual resource IDs, SSH and ping evidence, and post-cleanup state.

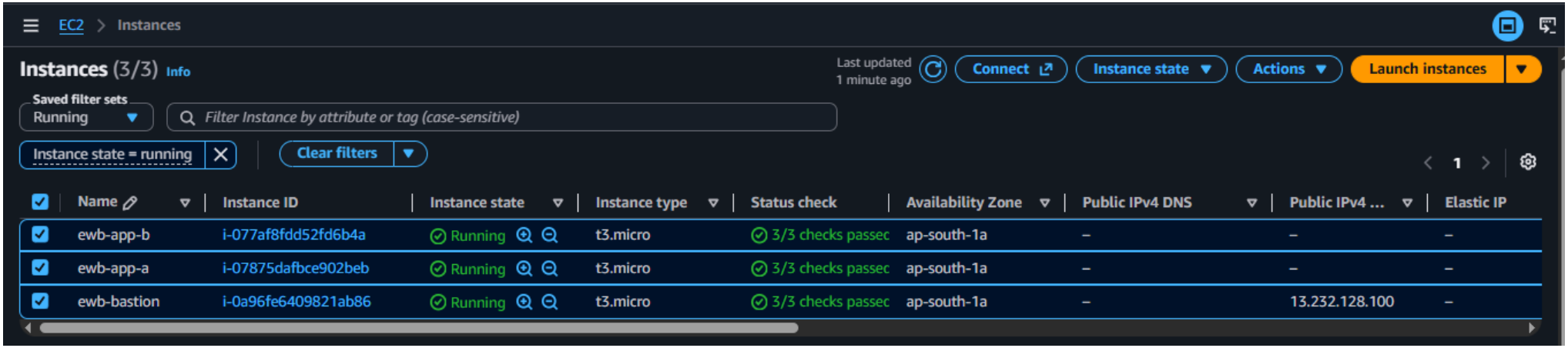
8. Cleanup and Final State

The supplied cleanup screenshots show all three project instances in Terminated state: ewb-bastion (i-0a96fe6409821ab86), ewb-app-a (i-07875dafbce902beb) and ewb-app-b (i-077af8fdd52fd6b4a). The VPCs screenshot shows one default VPC, vpc-068d3f58004814661 (172.31.0.0/16), in the displayed filtered list; the two named project VPCs are absent.

The provided archive does not contain a post-cleanup EBS volumes page, Elastic IP allocations page, or Billing view. Therefore, the report does not claim that all potentially chargeable resources are absent or that the final AWS bill is zero. A final manual check of lab EBS volumes/snapshots, Elastic IP allocations and AWS Billing remains prudent. Unrelated pre-existing practice resources should not be deleted as part of this lab cleanup.

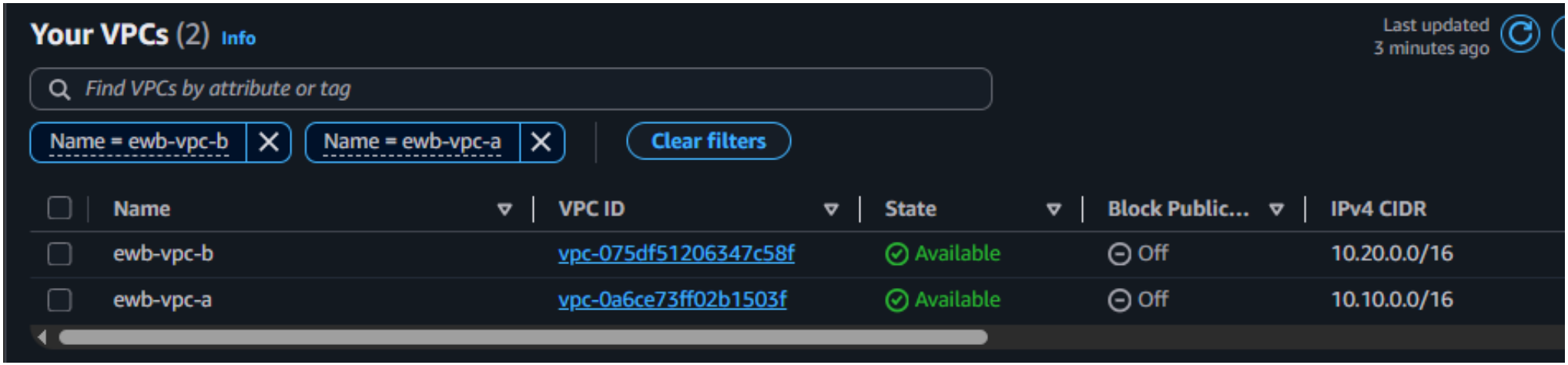
Evidence 01 — Instance inventory before cleanup

Host: AWS EC2 Console | Test: Instance inventory before cleanup | Result: All three t3.micro instances are Running in ap-south-1a at test time; only the bastion shows a public IPv4.



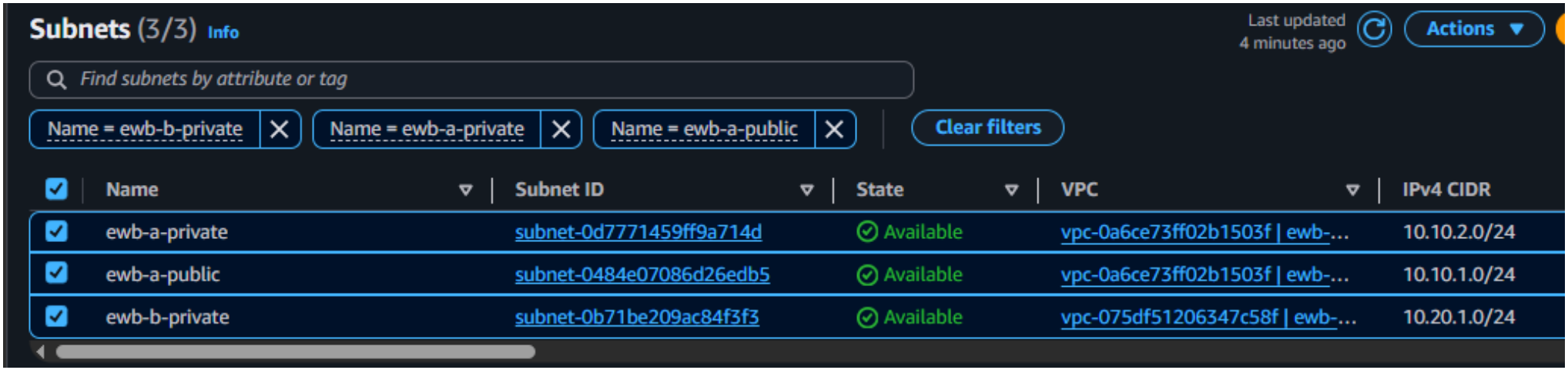
Evidence 02 — VPC CIDR inventory

Host: AWS VPC Console | Test: VPC CIDR inventory | Result: VPC A 10.10.0.0/16 and VPC B 10.20.0.0/16 are Available; IDs are shown.



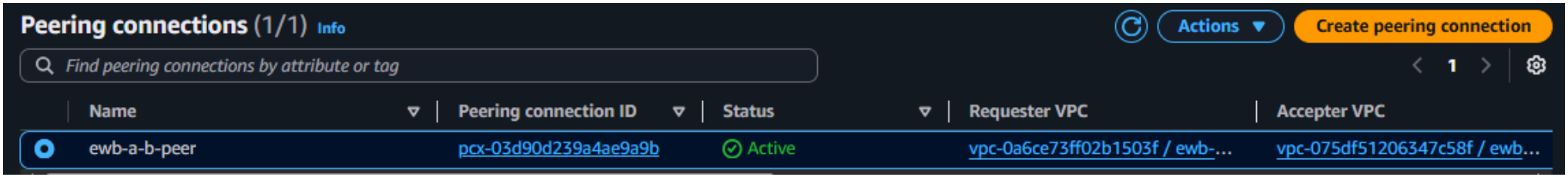
Evidence 03 — Subnet CIDR inventory

Host: AWS VPC Console | Test: Subnet CIDR inventory | Result: The three named subnets and their subnet IDs/CIDRs are shown.



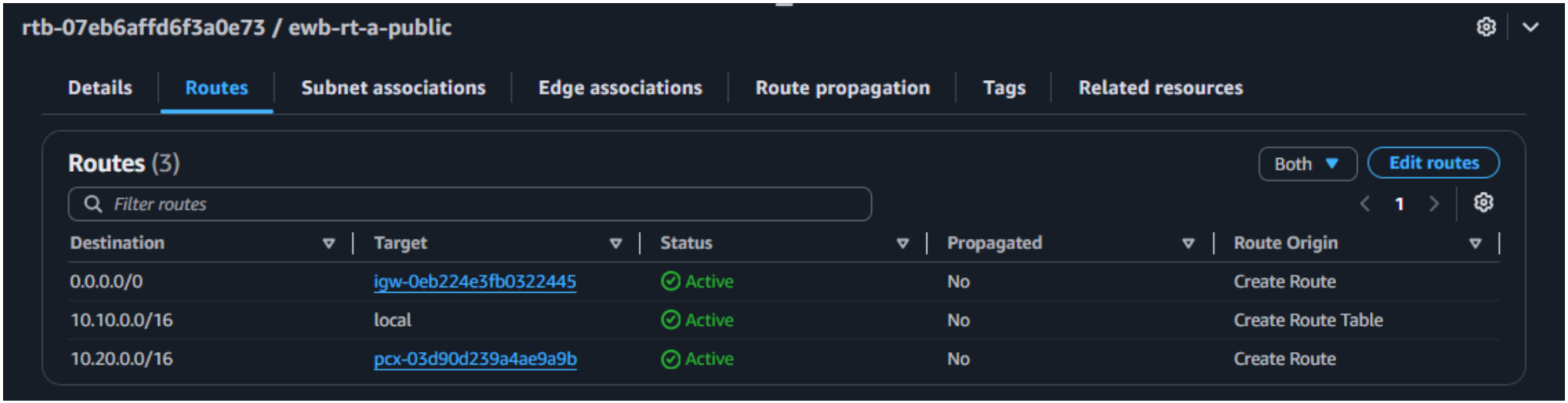
Evidence 04 — VPC peering validation

Host: AWS VPC Console | Test: VPC peering validation | Result: ewb-a-b-peer is Active with peering ID pcx-03d90d239a4ae9a9b.



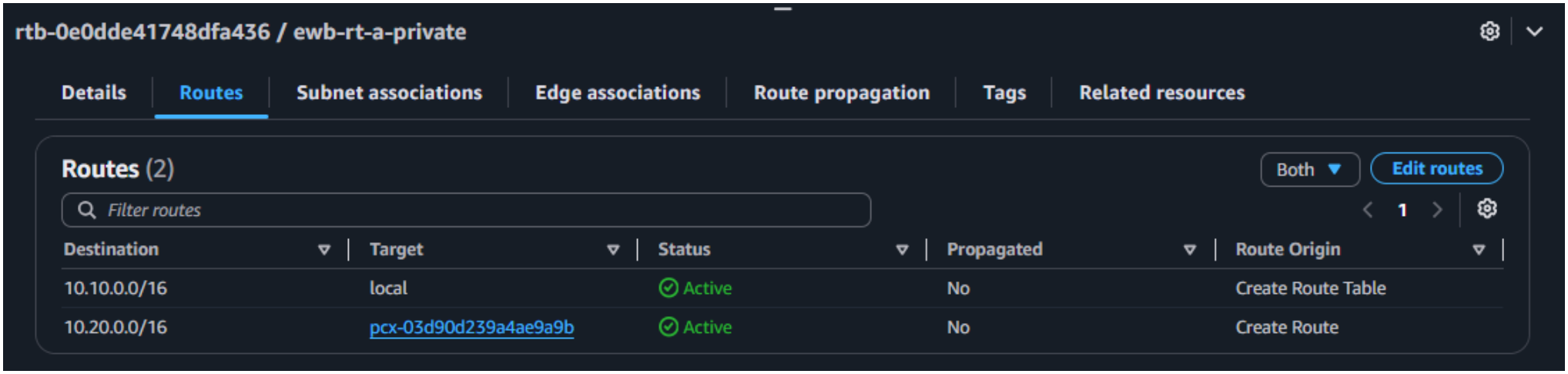
Evidence 05 — Public route table A

Host: VPC route table console | Test: Public route table A | Result: Local route, default route to IGW and peer-CIDR route are Active.



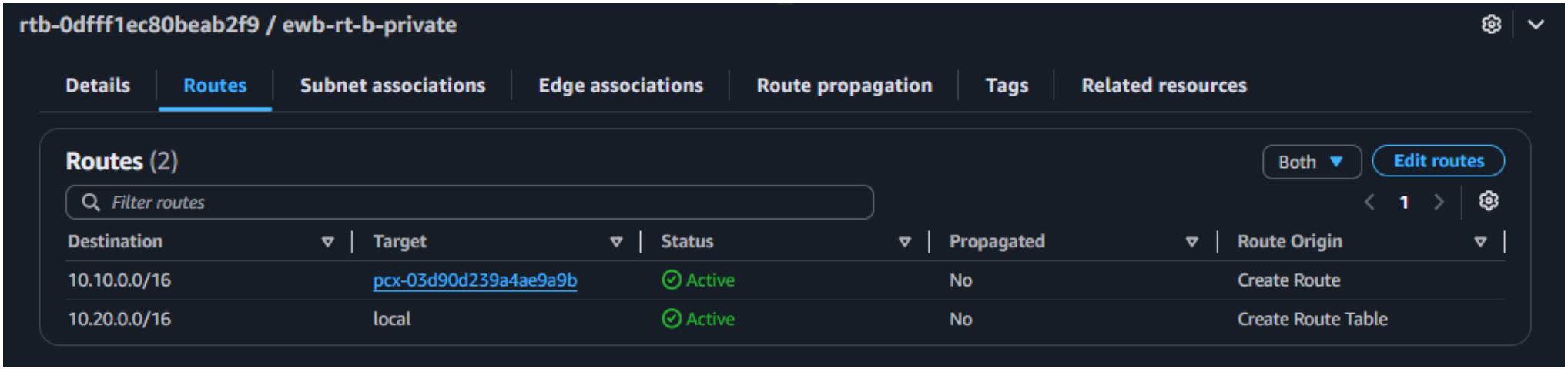
Evidence 06 — Private route table A

Host: VPC route table console | Test: Private route table A | Result: Local route and 10.20.0.0/16 peering route are Active.



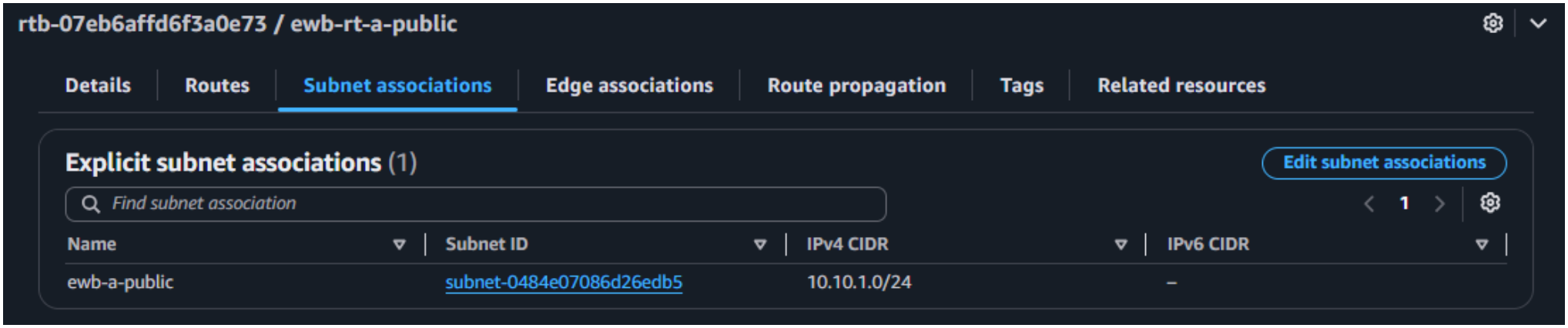
Evidence 07 — Private route table B

Host: VPC route table console | Test: Private route table B | Result: Local route and 10.10.0.0/16 peering route are Active.



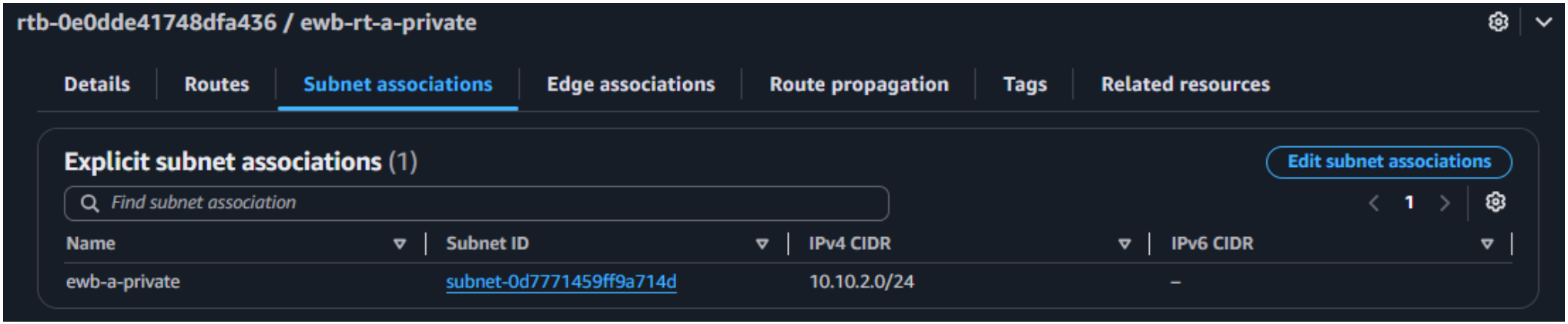
Evidence 08 — Public subnet association

Host: VPC route table console | Test: Public subnet association | Result: ewb-a-public is explicitly associated with ewb-rt-a-public.



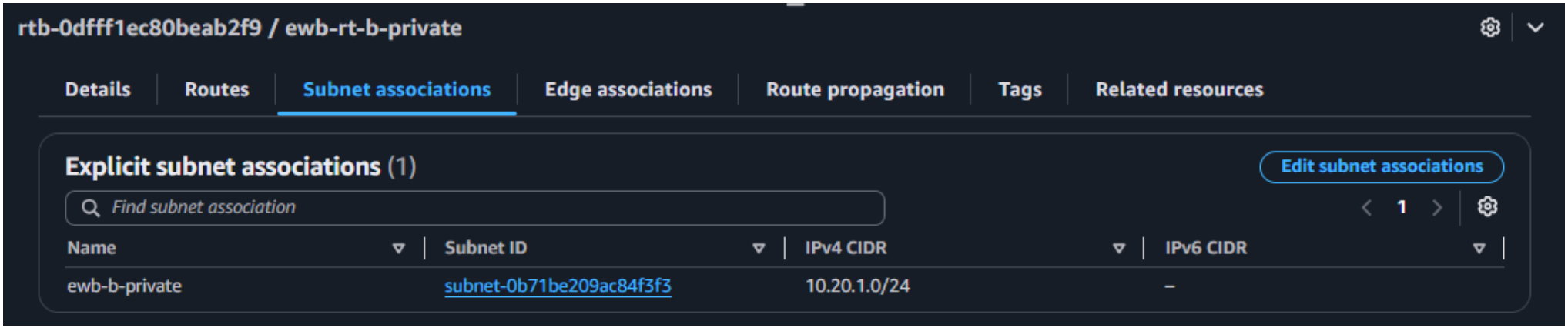
Evidence 09 — Private A subnet association

Host: VPC route table console | Test: Private A subnet association | Result: ewb-a-private is explicitly associated with ewb-rt-a-private.



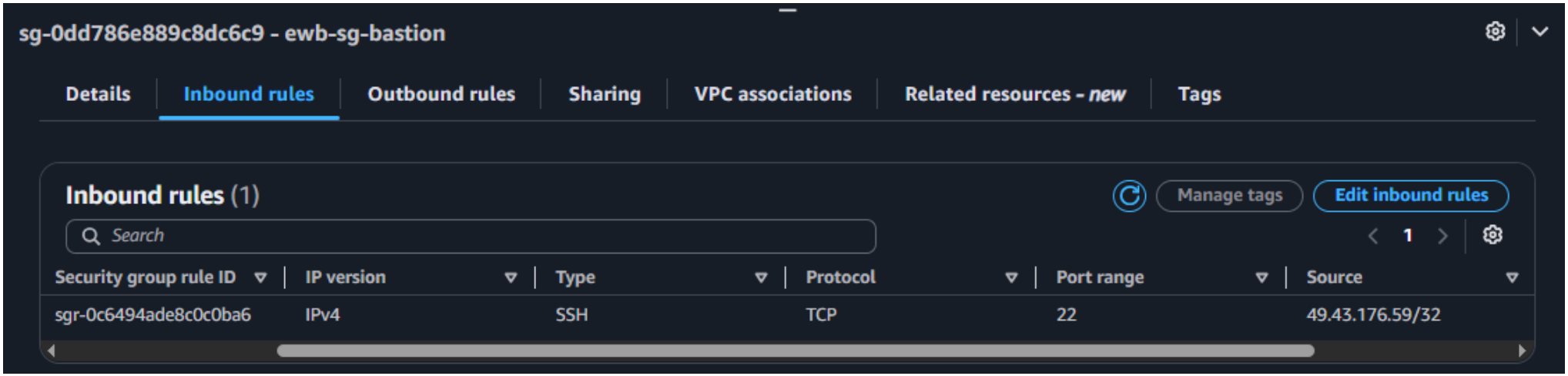
Evidence 10 — Private B subnet association

Host: VPC route table console | Test: Private B subnet association | Result: ewb-b-private is explicitly associated with ewb-rt-b-private.



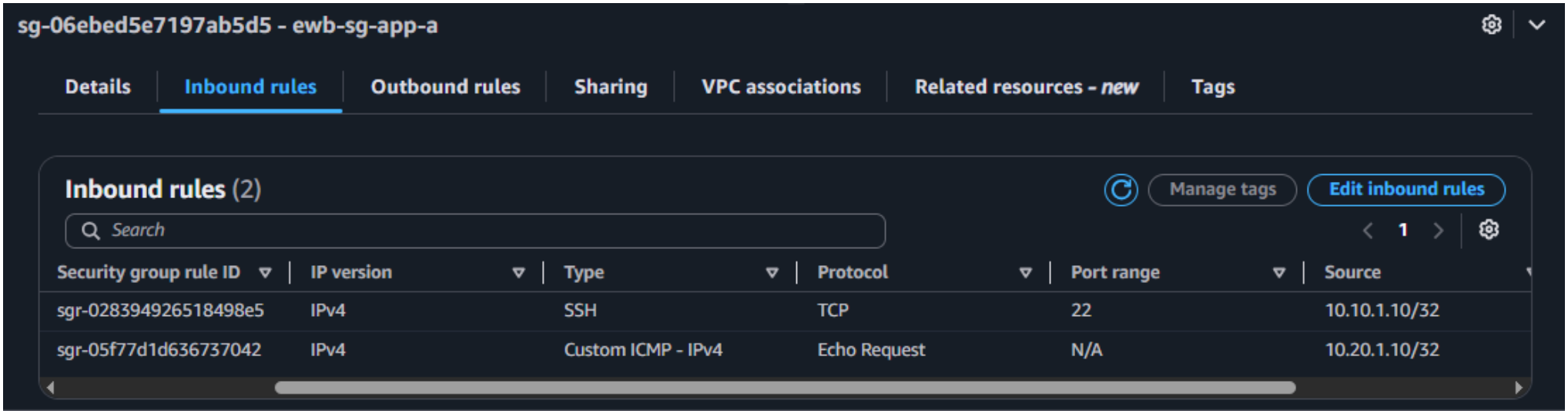
Evidence 11 — Bastion inbound SSH rule

Host: EC2 security groups | Test: Bastion inbound SSH rule | Result: SSH TCP/22 source is restricted to 49.43.176.59/32 in the captured configuration.



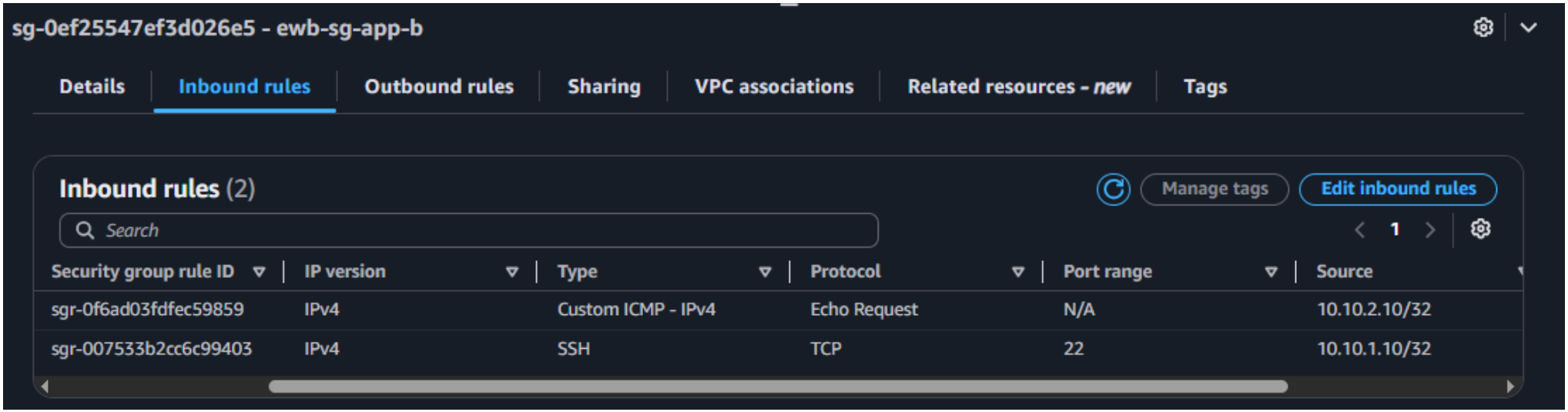
Evidence 12 — App-A SSH and ICMP rules

Host: EC2 security groups | Test: App-A SSH and ICMP rules | Result: SSH TCP/22 from 10.10.1.10/32 and ICMP Echo Request from 10.20.1.10/32.



Evidence 13 — App-B SSH and ICMP rules

Host: EC2 security groups | Test: App-B SSH and ICMP rules | Result: SSH TCP/22 from 10.10.1.10/32 and ICMP Echo Request from 10.10.2.10/32.



Evidence 14 — SSH to bastion

Host: Windows PowerShell / Amazon Linux 2023 | Test: SSH to bastion | Result: ssh ewb-bastion succeeds; hostname and 10.10.1.10 address are shown.

```
PS D:\download> ssh ewb-bastion
The authenticity of host '13.232.128.100 (13.232.128.100)' can't be established.
ED25519 key fingerprint is SHA256:6X5PxypfnLKlCGmnK89HJUGDcHaP894ortXNiu8Ewrk.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '13.232.128.100' (ED25519) to the list of known hosts.
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-10-10-1-10 ~]$ hostname
ip-10-10-1-10.ap-south-1.compute.internal
[ec2-user@ip-10-10-1-10 ~]$ ip -4 addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    inet          /8 scope host lo
       valid_lft forever preferred_lft forever
2: ens5: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9001 qdisc mq state UP group default qlen 1000
    altname enp0s5
    altname eni-0a21e350ebacd6782
    altname device-number-0.0
    inet           /24 metric 512 brd            scope global dynamic ens5
       valid_lft 3338sec preferred_lft 3338sec
[ec2-user@ip-10-10-1-10 ~]$ exit
logout
Connection to 13.232.128.100 closed.
```

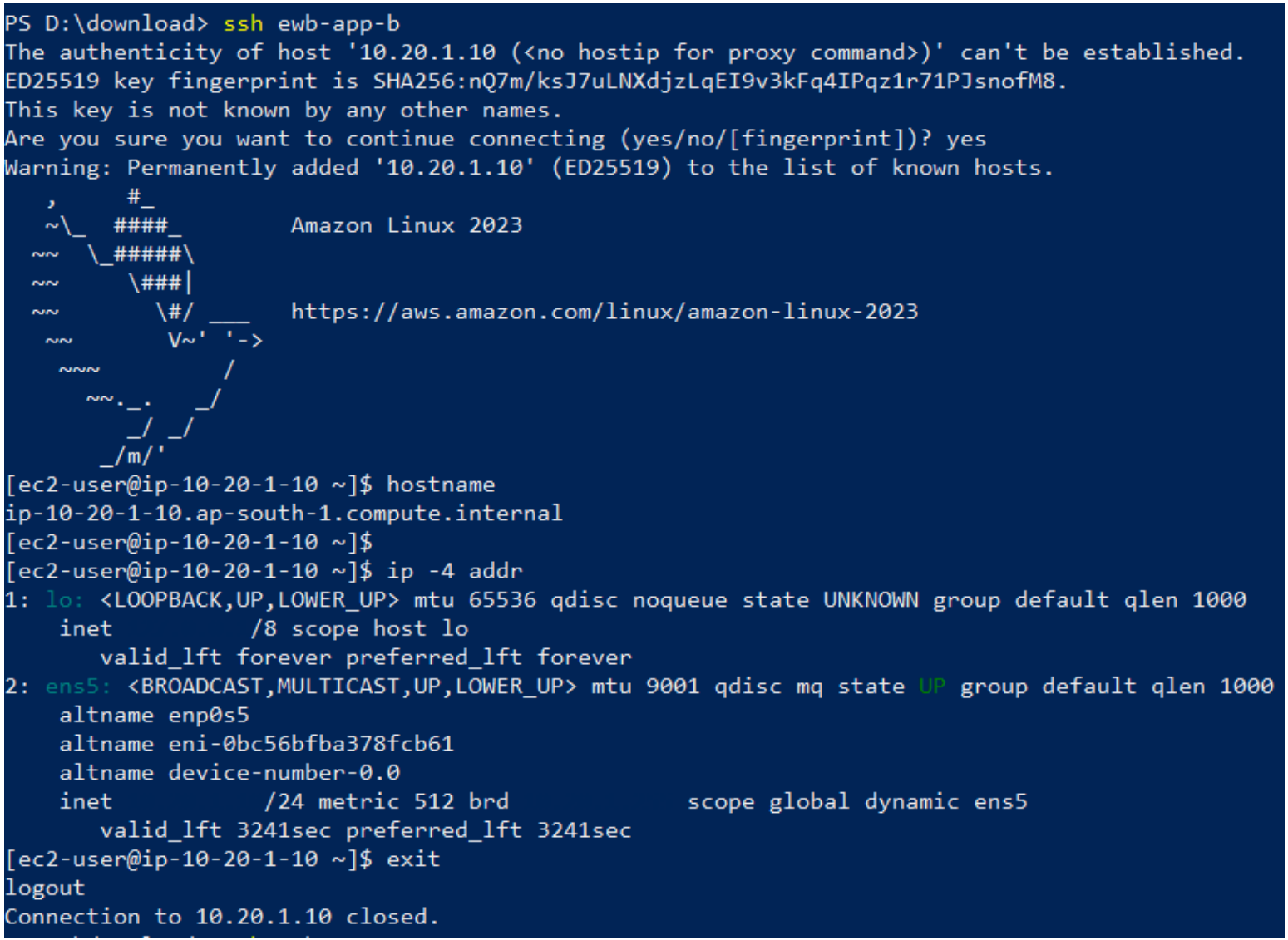
Evidence 15 — Private SSH to App-A

Host: Windows PowerShell / Amazon Linux 2023 | Test: Private SSH to App-A | Result: ssh ewb-app-a succeeds through ProxyJump; private address is 10.10.2.10.

```
PS D:\download> ssh ewb-app-a
The authenticity of host '10.10.2.10 (<no hostip for proxy command>)' can't be established.
ED25519 key fingerprint is SHA256:XVAfxRTU8TGn43r8Z4AMc4cIO4yEnBkrPhvcdcs85fQ.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.10.2.10' (ED25519) to the list of known hosts.
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-10-10-2-10 ~]$ hostname
ip-10-10-2-10.ap-south-1.compute.internal
[ec2-user@ip-10-10-2-10 ~]$ ip -4 addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    inet          /8 scope host lo
       valid_lft forever preferred_lft forever
2: ens5: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9001 qdisc mq state UP group default qlen 1000
    altname enp0s5
    altname eni-05586acf11a5f4872
    altname device-number-0.0
    inet           /24 metric 512 brd            scope global dynamic ens5
       valid_lft 3283sec preferred_lft 3283sec
[ec2-user@ip-10-10-2-10 ~]$ exit
logout
Connection to 10.10.2.10 closed.
```

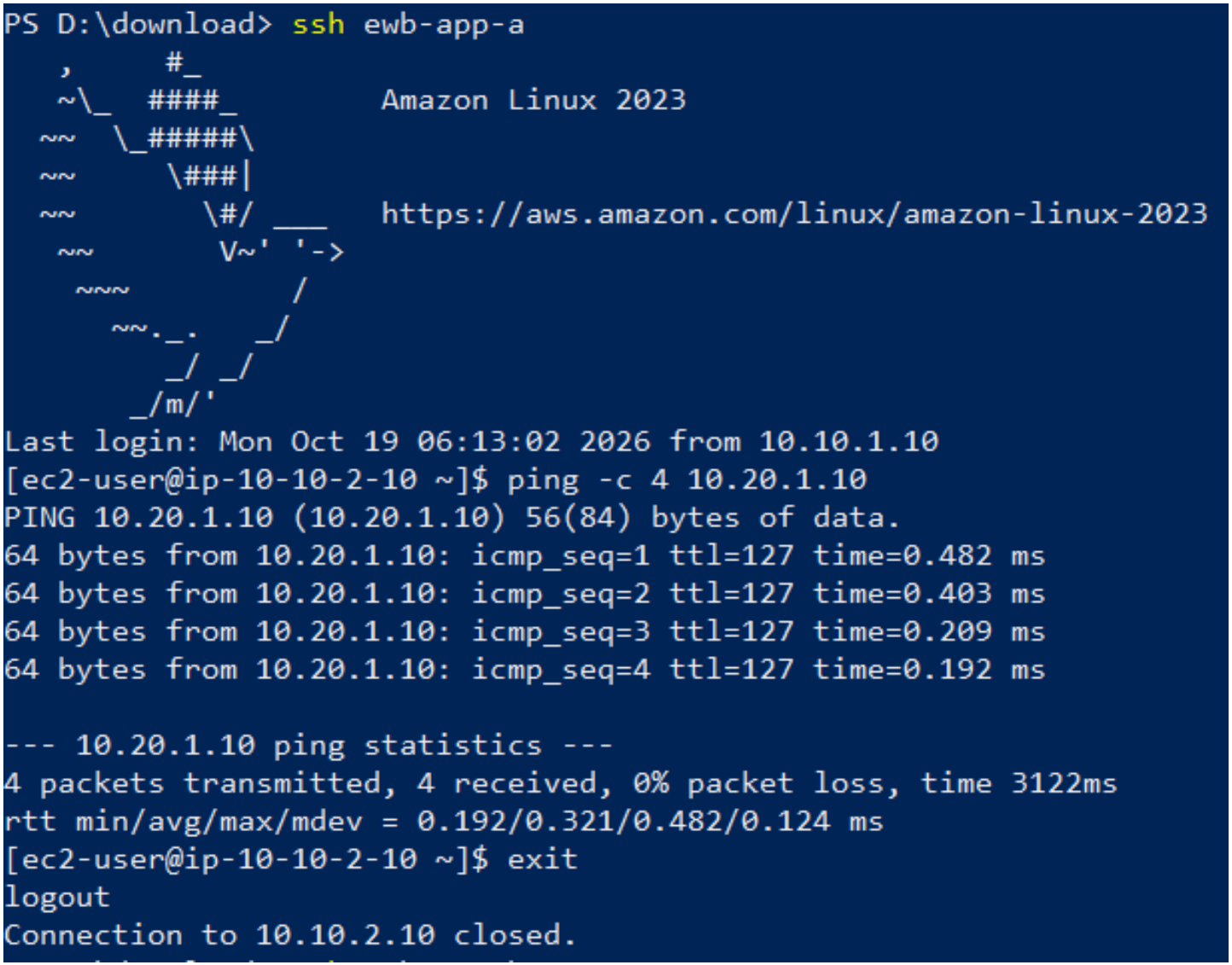
Evidence 16 — Private SSH to App-B

Host: Windows PowerShell / Amazon Linux 2023 | Test: Private SSH to App-B | Result: ssh ewb-app-b succeeds through ProxyJump; private address is 10.20.1.10.



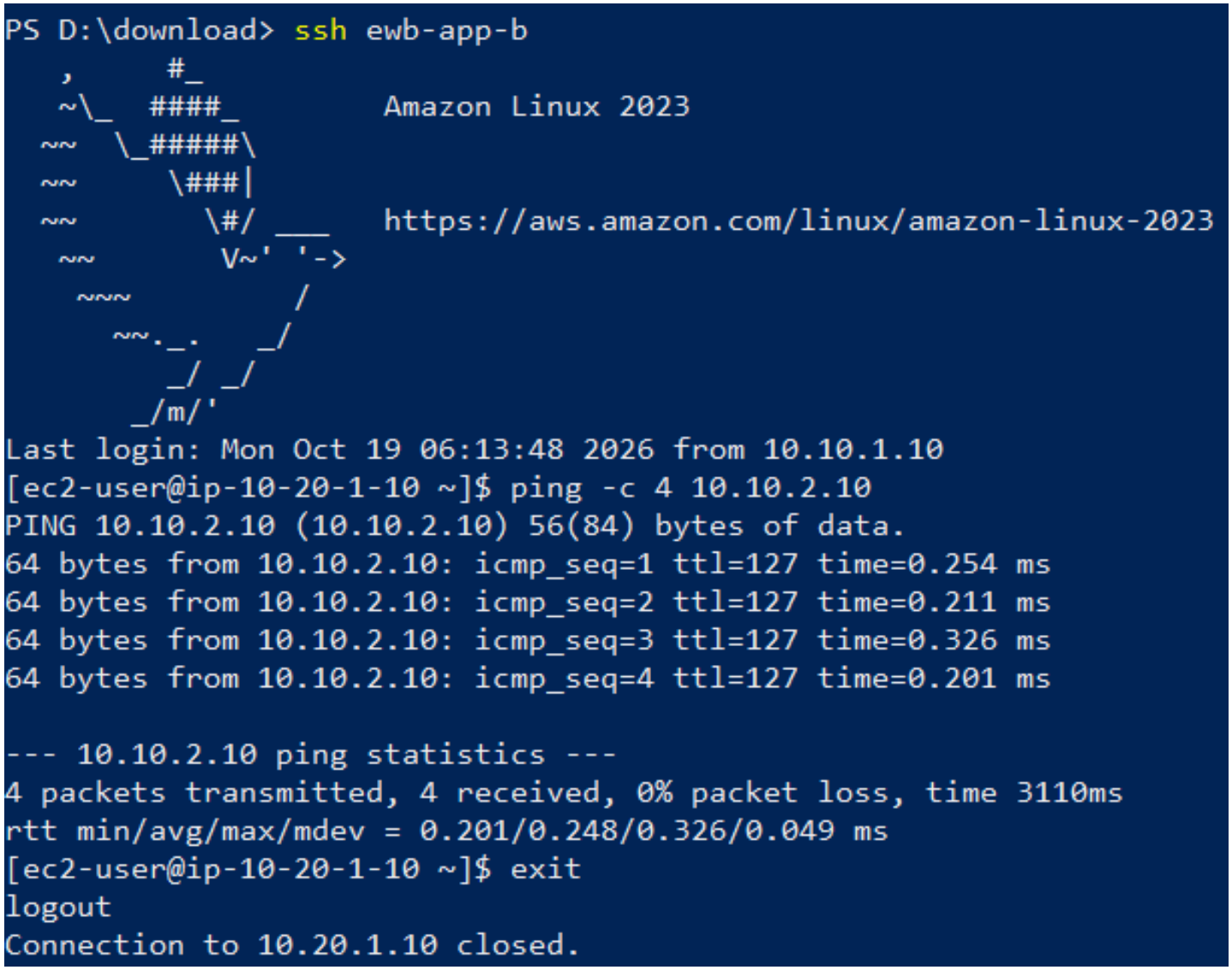
Evidence 17 — Forward private ICMP test

Host: App-A shell | Test: Forward private ICMP test | Result: App-A → 10.20.1.10 receives 4/4 replies with 0% packet loss.



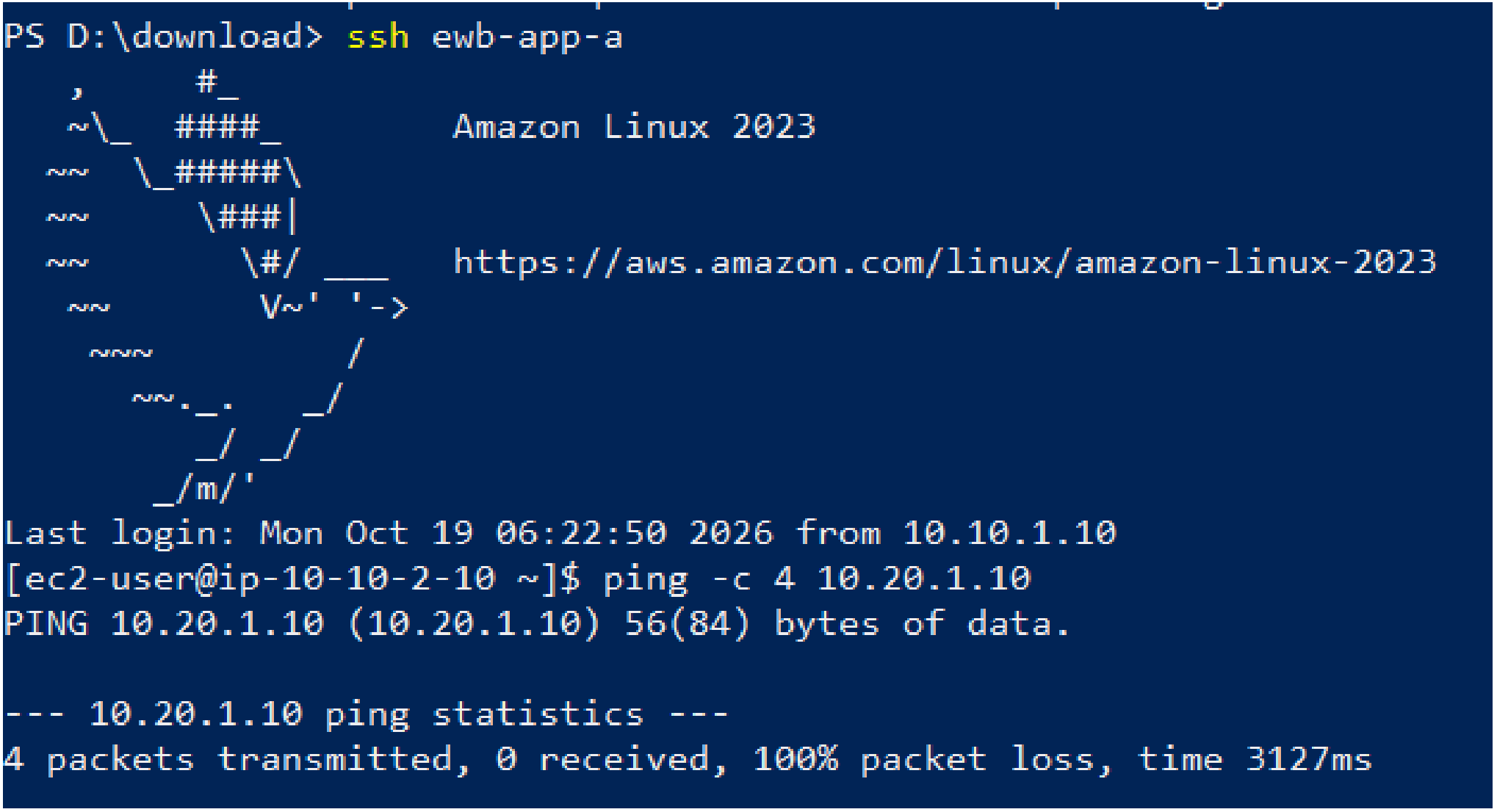
Evidence 18 — Reverse private ICMP test

Host: App-B shell | Test: Reverse private ICMP test | Result: App-B → 10.10.2.10 receives 4/4 replies with 0% packet loss.



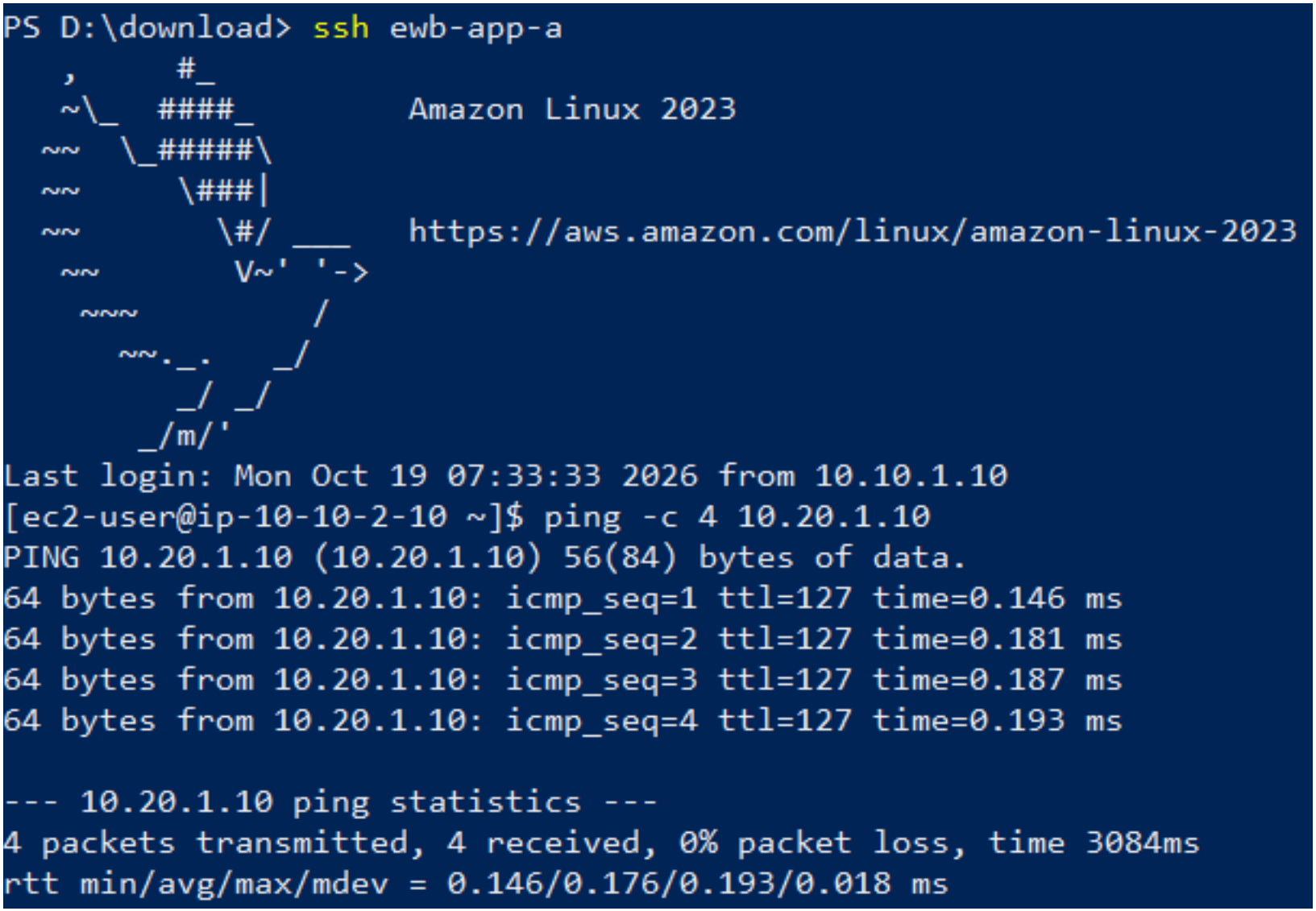
Evidence 19 — Controlled ICMP failure

Host: App-A shell / AWS security group | Test: Controlled ICMP failure | Result: After removing App-B inbound Echo Request from 10.10.2.10/32, the ping shows 0 received and 100% packet loss.



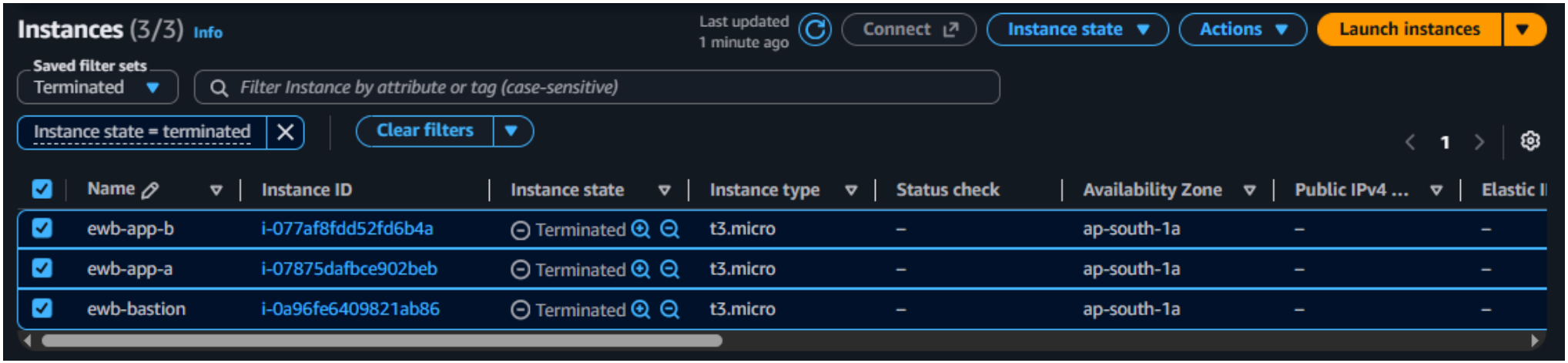
Evidence 20 — ICMP recovery

Host: App-A shell / AWS security group | Test: ICMP recovery | Result: After restoring the App-B Echo Request rule, the ping returns to 4/4 replies and 0% packet loss.



Evidence 21 — Post-cleanup instance status

Host: AWS EC2 Console | Test: Post-cleanup instance status | Result: All three project EC2 instances show Terminated.



Evidence 22 — Post-cleanup VPC inventory

Host: AWS VPC Console | Test: Post-cleanup VPC inventory | Result: The displayed filtered list contains only the default VPC vpc-068d3f58004814661; project VPCs are absent.

